



廈門工學院

XIAMEN INSTITUTE OF TECHNOLOGY



Private University

单片机应用课程设计 课程设计报告

题 目： 循环流水灯计数器设计

专业班级： 物联网 1 班

学 号： 2302140130

姓 名： 聂子明

指导教师： 苏河星

分 数：

人工智能学院

循环流水灯计数器设计

摘要:

本课程设计基于 51 单片机实现循环流水灯计数器，核心任务包括设计单片机最小系统、LED 驱动电路、数码管显示电路及独立按键控制电路。硬件上采用 STC89C52RC 单片机，搭配 12MHz 晶振时钟电路、上电复位电路，通过 P1 口驱动 8 个 LED（限流电阻 330 Ω ），利用动态扫描驱动共阴极数码管显示计数（0-99 循环）和速度档位（1 - 慢、2 - 中、3 - 快），P3.2 引脚连接按键实现消抖及速度调节。软件通过定时器中断实现数码管动态扫描和流水灯定时控制，状态机处理按键短按 / 长按事件。经测试，系统稳定实现流水灯循环、计数显示及三档速度切换功能，具备一定的工程实践参考价值，后续可扩展方向切换、花样显示等功能以提升交互性。

目 录

摘要	I
第 1 章 概述	4
1.1 设计背景及意义	4
1.2 设计内容及方法	4
第 2 章 系统分析	6
2.1 可行性分析	6
2.2 功能需求分析	6
2.3 性能需求分析	6
第 3 章 设计方案及设计原理	7
3.1 设计思路	7
3.2 设计原理	8
第 4 章 硬件设计	9
4.1 硬件设计方案	9
4.2 单片机模块	9
4.3 流水灯模块	10
4.4 数字显示模块	10
4.5 按键模块	10
第 5 章 软件设计	11
5.1 系统架构设计	11
5.2 声明、定义及主程序设计	11
5.3 延时函数设计	13
5.4 数字显示模块设计	14
5.5 按键模块设计	15
5.6 流水灯模块设计	16
5.7 定时器初始化与中断服务函数设计	17
第 6 章 功能实现	18
6.1 系统功能实现	18
6.2 数字显示功能实现	18
6.3 按键功能实现	20
6.4 流水灯功能实现	23
第 7 章 系统测试	25
7.1 测试目的	25

7.2 功能测试	25
结论	27
参考文献	28
附录 A	29

第 1 章 概述

1.1 设计背景及意义

1.1.1 设计背景

随着电子技术的飞速发展，单片机作为嵌入式系统的核心部件，在工业控制、智能家居、消费电子等领域得到广泛应用。循环流水灯计数器作为单片机入门级的经典应用，是理解单片机 I/O 口控制、定时器中断、数码管显示等基础功能的重要载体。通过设计该系统，不仅能够掌握单片机最小系统搭建、硬件电路设计与调试方法，还能加深对软件编程逻辑、人机交互设计的理解，为后续复杂嵌入式系统开发奠定基础。

1.1.2 设计意义

本次循环流水灯计数器设计将理论知识与实践操作紧密结合，对巩固单片机原理与应用课程的学习成果具有重要意义。从技术层面看，它涵盖了硬件电路设计、软件编程、系统调试等全流程开发，有助于提升综合实践能力；从工程应用角度，该设计为实现更复杂的 LED 显示控制、实时计数监测等功能提供了技术原型，可作为交通信号灯控制、电子广告牌等项目的设计参考，体现了单片机在实际场景中的灵活性与实用性。

1.2 设计内容及方法

1.2.1 设计内容

本设计围绕 51 单片机展开，旨在实现循环流水灯计数器功能。硬件设计上，构建单片机最小系统，包括电源电路、时钟电路和复位电路，为系统稳定运行提供基础；搭建 8 个 LED 驱动电路，合理选择 330Ω 限流电阻保障 LED 正常工作，呈现稳定流水灯效果；采用动态扫描方式驱动共阴极数码管，通过 P0 口控制段选、P2 口控制位选，实时显示流水灯移动次数与速度档位；设计独立按键电路连接 P3.2 引脚，结合状态机进行消抖处理，实现流水灯速度调节、启停控制等功能。软件设计涵盖定时器中断、按键处理、数码管显示等模块，协同完成各项功能。

1.2.2 设计方法

硬件设计采用自顶向下的方法，先确定单片机最小系统框架，再逐步细化各功能模块

电路。通过理论计算与实际选型，确定元器件参数，如根据 LED 工作特性计算限流电阻阻值；运用 Proteus 等工具进行电路仿真，验证设计可行性。

软件设计运用模块化编程思想，将系统划分为定时器中断、按键处理、流水灯控制等功能模块，分别编写代码实现对应功能。定时器 0 配置为模式 1 产生 1ms 中断，驱动数码管动态扫描与流水灯定时；按键处理采用状态机消除抖动，区分短按、长按操作；主程序通过循环检测按键状态，调用各模块函数实现系统功能，并通过全局变量实现模块间数据交互与状态同步。

第 2 章 系统分析

2.1 可行性分析

硬件方面，51 单片机资源丰富、外设接口成熟，搭配电源、时钟、复位电路构成的最小系统稳定可靠；LED 驱动电路通过合理选择限流电阻可保障安全工作，动态扫描数码管驱动方式能有效减少 I/O 口占用，独立按键消抖设计成熟，整体硬件设计可行。

软件层面，模块化编程结合定时器中断、状态机等技术，可实现流水灯控制、数码管动态显示、按键处理等功能，代码逻辑清晰，易于实现与维护。

2.2 功能需求分析

利用 51 单片机设计和实现一款循环流水灯计数器。要求：

(1) 选择合适的单片机型号（如 51 单片机），设计单片机最小系统，包括电源电路、时钟电路和复位电路。

(2) 设计 8 个 LED 的驱动电路，合理选择限流电阻，保证 LED 正常发光且不被损坏，同时考虑驱动方式，确保流水灯效果稳定、流畅。

(3) 采用动态扫描或静态驱动方式驱动数码管，根据数码管类型（共阳或共阴）设计相应电路，保证数码管清晰、准确地显示流水灯计数和速度档位等信息。

(4) 设计独立按键电路，实现流水灯速度控制和扩展功能（如方向、花样切换）的按键操作，注意按键的消抖处理，提高按键操作的可靠性。

2.3 性能需求分析

性能上，系统通过定时器精准控制流水灯速度，动态扫描保证数码管无闪烁显示，按键消抖提升操作可靠性，能稳定满足设计要求。

成本方面，51 单片机及周边元器件价格适中，电路设计简单，无需复杂加工工艺，开发成本较低，适合小型项目使用。

第3章 设计方案及设计原理

3.1 设计思路

本次循环流水灯计数器设计以功能需求为导向，从硬件和软件两方面协同推进。硬件上，先搭建 51 单片机最小系统确保核心稳定运行，再围绕 LED、数码管、按键等功能模块，精准设计驱动与控制电路；软件层面，采用模块化编程，利用定时器中断实现数码管动态扫描与流水灯定时，状态机处理按键消抖及功能切换，最终实现稳定的交互功能。

(1) 硬件设计思路：选用 STC89C52RC 单片机，搭建由电源、时钟、复位电路组成的最小系统，保障芯片稳定工作。设计 LED 驱动电路时，根据 LED 参数计算 330Ω 限流电阻，通过 P1 口直接驱动 8 个 LED 实现流水效果。数码管采用动态扫描驱动方式，共阴极数码管段选连接 P0 口、位选连接 P2 口，通过分时复用实现计数与速度档位显示。独立按键连接 P3.2 引脚，结合电容与软件消抖设计，提升按键操作可靠性。

(2) 软件设计思路：采用模块化编程思想，将系统划分为定时器中断、按键处理、流水灯控制、数码管显示等模块。定时器 0 配置为模式 1，定时 1ms 产生中断，在中断服务函数中实现数码管动态扫描，并根据流水灯运行状态与速度档位控制 LED 切换。按键处理模块通过状态机区分按键短按、长按，实现流水灯启停与速度调节。主程序循环检测按键状态，更新显示缓冲区数据，各模块通过全局变量实现数据交互，确保系统功能协同运行。

基于 STC89C52RC 单片机的循环流水灯计数器设计如图 3-1 所示。

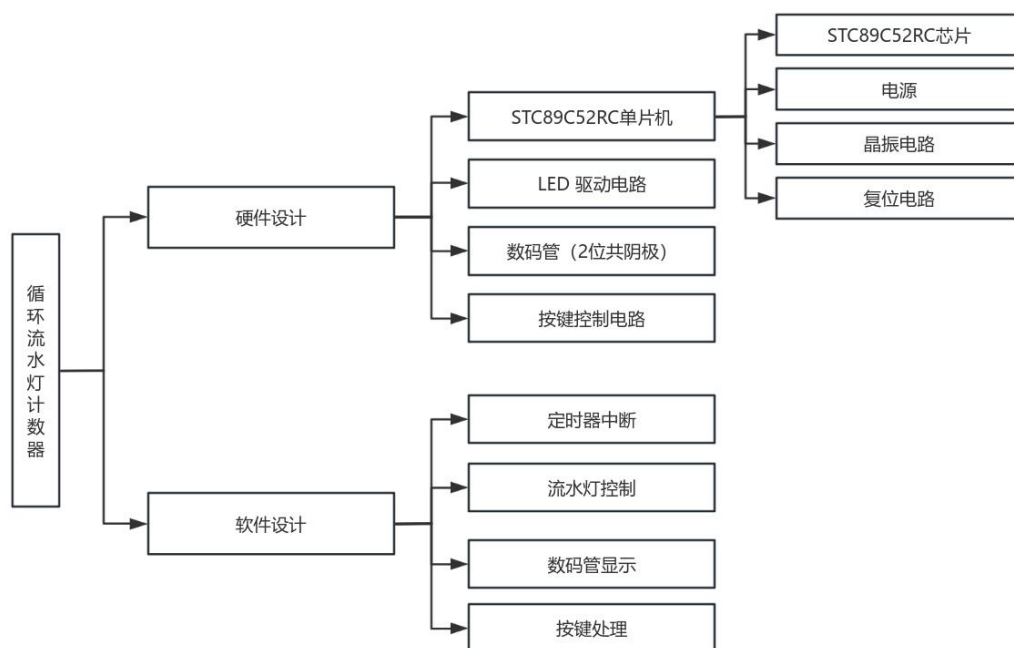


图 3-1 循环流水灯计数器设计图

3.2 设计原理

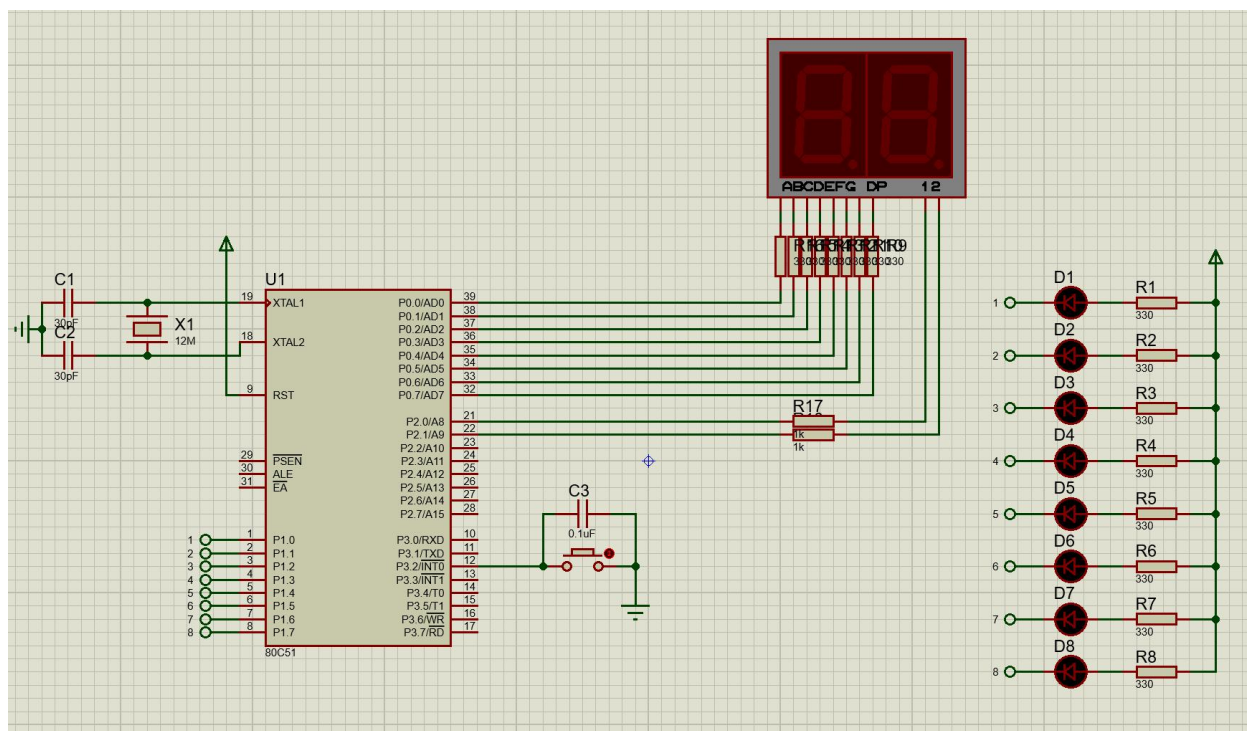


图 3-2 循环流水灯原理图

第4章 硬件设计

4.1 硬件设计方案

本课程设计的循环流水灯计数器方案以 STC89c52RC 单片机为核心。按键识别模块捕捉用户动作指令，转化为电信号传输至单片机，实现流水灯操控功能；数码管对流水灯流动次数进行记数。

按键模块为用户提供手动操作方式，用户通过按键输入指令，按键识别模块捕捉用户动作指令，转化为电信号传输至单片机，单片机接收并处理，实现流水灯操控功能。

数码管对流水灯流动次数进行记数，转化模式时显示当前运行情况对应代表代码，方便用户直观了解运行情况。

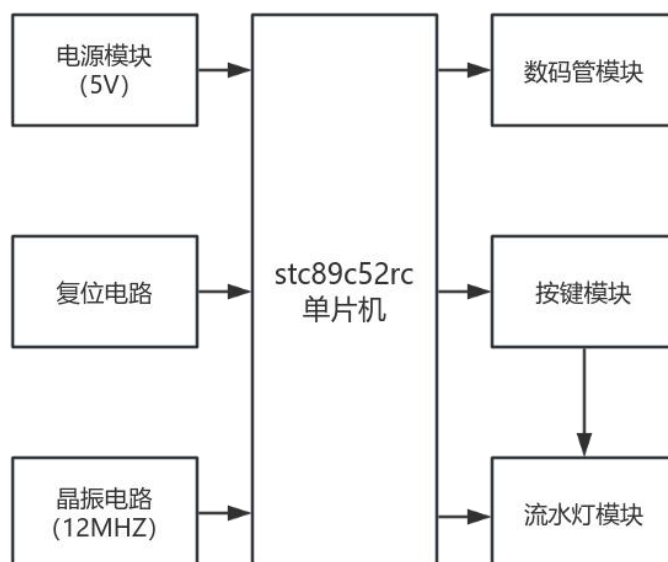


图 4-1 系统整体构架图

4.2 单片机模块

选择 STC89C52RC 单片机作为控制核心，其最小系统包括以下部分：

- (1) 电源电路：采用 + 5V 直流电源供电，通过电容滤波确保电源稳定。
- (2) 时钟电路：使用 12MHz 晶振和两个 30pF 电容组成时钟电路，为单片机提供稳定的时钟信号。
- (3) 复位电路：采用上电复位和手动复位相结合的方式，确保系统可靠复位。

4.3 流水灯模块

(1) LED 选择：选用普通多色 LED 发光二极管，正向压降约为 2.0V，额定电流为 20mA。

(2) 限流电阻计算：电源电压为 5V，LED 正向压降为 2.0V，限流电阻计算公式为：

$$R = \frac{V_{CC} - V_F}{I_F} = \frac{5V - 2V}{10mA} = 300 \Omega \quad 4.1 \text{ 限流电阻计算公式}$$

实际选用 330 Ω 电阻，确保 LED 工作在安全电流范围内。

(3) 驱动方式：采用单片机 I/O 口直接驱动，通过 P1 口控制 8 个 LED。

4.4 数字显示模块

(1) 数码管选择：选用共阴极两位数码管，段选信号通过 P0 口控制，位选信号通过 P2 口控制。

(2) 驱动方式：采用动态扫描方式驱动数码管，通过定时器控制扫描频率，确保显示稳定无闪烁。

(3) 保护电路：阳极（a, b, c, d, e, f, g, dp）串联 330 Ω 电阻，阴极（1, 2）串联 1k Ω 电阻确保数码管工作在安全电流范围内。

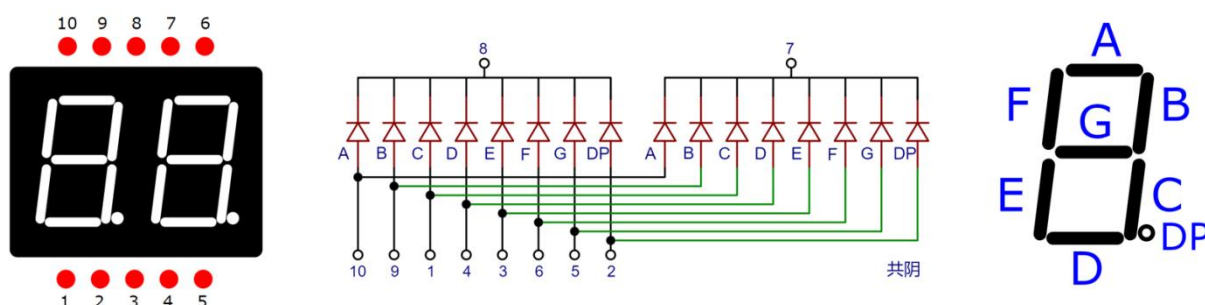


图 4-2-2 2 位共阴极数码管结构展示图

4.5 按键模块

(1) 按键选择：使用轻触按键，连接到 P3.2 引脚。

(2) 消抖处理：

① 软件消抖：通过延时和状态机实现按键消抖，提高按键操作的可靠性。

② 硬件消抖：并联 0.1 μF 陶片电容，如有抖动短时间内持续提供供电，起到防抖效果。

第 5 章 软件设计

5.1 系统架构设计

采用定时器中断驱动的方式实现流水灯和数码管的动态显示，主循环负责按键检测和状态处理。系统主要模块包括：

- (1) 定时器中断服务模块：负责数码管扫描显示和流水灯定时控制。
- (2) 按键处理模块：负责按键检测和消抖处理，实现速度控制和功能切换。
- (3) 流水灯控制模块：负责控制 LED 的点亮和熄灭，实现流水效果。
- (4) 数码管显示模块：负责将计数和状态信息显示在数码管上。

5.2 声明、定义及主程序设计

5.2.1 头文件与引脚定义

```
1 //定义头文件
2 #include<reg52.h>           // 包含8051单片机寄存器定义
3 #include<intrins.h>         // 包含特殊函数的头文件
4
5 // 定义端口
6 #define LED_PORT P1         // 8个LED连接到P1口
7 #define SEG_PORT P0         // 数码管段选连接到P0口
8 #define BIT_PORT P2         // 数码管位选连接到P2口
9
10 // 定义按键引脚
11 sbit KEY_SPEED = P3^2;     // 速度控制按键连接到P3.2
```

图 5-1 头文件与引脚定义（代码）

- (1) reg52.h: 包含 8051 内核的特殊功能寄存器（SFR）定义；
- (2) intrins.h: 提供空操作（nop）等底层函数；
- (3) 端口定义采用宏定义方式，便于程序移植。

5.2.2 函数声明及全局变量定义

```

13 // 函数原型声明
14 void delay_ms(unsigned int ms); //延时函数（定义为毫秒级延时）
15 void display_no_delay(void); //数码管动态扫描显示函数
16 unsigned char key_process(void); //按键检测与处理函数
17 void led_flow_no_delay(void); //流水灯控制函数
18 void Timer0_Init(void); //定时器 0 初始化函数
19 void Timer0_ISR(void); //定时器 0 中断服务函数
20 void show_status_code(unsigned char code1, unsigned char code2); //显示状态码（开始/暂停；速度）
21
22 // 定义全局变量
23 unsigned char led_pos = 0; // 当前LED位置(从0开始)
24 unsigned char count = 0; // 流水灯计数(从0开始)
25 unsigned char speed_level = 2; // 默认中速(1-慢,2-中,3-快)
26 unsigned int delay_time[] = {500, 300, 150}; // 不同速度对应的延时时间(ms)
27 unsigned char code seg_table[] = { // 共阴极数码管段码表
28     0x3F, 0x06, 0x5B, 0x4F, // 0-3
29     0x66, 0x6D, 0x7D, 0x07, // 4-7
30     0x7F, 0x6F, 0x77, 0x7C, // 8-9,A-B
31     0x39, 0x5E, 0x79, 0x71 // C-F
32 };
33 bit isRunning = 0; // 流水灯默认停止
34 bit show_status = 0; // 是否显示状态标志
35 unsigned int status_timer = 0; // 状态显示计时器
36 bit isChangingSpeed = 0; // 是否正在进行速度调节
37 bit firstStart = 1; // 首次启动标志
38
39 // 数码管显示相关变量
40 unsigned char display_buffer[2] = {0, 0}; // 显示缓冲区
41 unsigned char display_pos = 0; // 当前显示位
42 unsigned int led_timer = 0; // LED流水灯定时器

```

图 5-2 函数声明及全局变量定义（代码）

5.2.3 主函数

```

217 // 主函数
218 void main() {
219     unsigned char key_result; //定义变量“key_result”储存按键处理结果
220
221     // 流水灯初始化
222     LED_PORT = 0xFF; // 熄灭所有LED
223     Timer0_Init(); // 初始化定时器
224
225     while(1) {
226         // 检测按键
227         key_result = key_process();
228
229         if(key_result == 1) { // 短按: 控制开始/暂停
230             isRunning = !isRunning; // 切换状态
231             if(isRunning) {
232                 show_status_code(0xC, 0xA); // 显示“AC”, “开始”标志
233             } else {
234                 show_status_code(0x0, 0x5); // 显示“SO”, “停止”标志
235                 // 因数码管无法直接显示“SO”, 用“50”代替
236             }
237         } else if(key_result == 2) { // 长按: 显示档位并开始流水灯
238
239             // 显示改变后的速度档位
240             show_status_code(speed_level, 0xA); // 显示“A”+速度等级
241
242             // 切换速度档位
243             speed_level++;
244             if(speed_level > 3) speed_level = 1;
245         }
246
247         // 如果不显示状态, 则正常显示计数
248         if(!show_status && !isChangingSpeed) {
249             display_buffer[0] = count % 10; // 显示个位
250             display_buffer[1] = count / 10; // 显示十位
251         }
252     }
253 }

```

图 5-3 主函数（代码）

5.3 延时函数设计

```

44 // 延时函数(ms级); 晶振: 12.000MHz
45 void delay_ms(unsigned int ms) {
46     unsigned int i, j; //定义整型变量“i”, “j”
47     for(i = 0; i < ms; i++)
48         for(j = 0; j < 100; j++); // 根据晶振频率设定为100 (12.000MHz)
49 }
50

```

图 5-4 延时函数（代码）

5.4 数字显示模块设计

```
51 // 数码管显示函数(无延时)
52 void display_no_delay() {
53     // 消隐
54     SEG_PORT = 0x00;
55     // 位选
56     switch(display_pos) {
57         case 0: BIT_PORT = 0x01; break; // 显示个位
58         case 1: BIT_PORT = 0x02; break; // 显示十位
59     }
60     // 段选
61     SEG_PORT = seg_table[display_buffer[display_pos]];
62
63     // 更新显示位置
64     display_pos++;
65     if(display_pos >= 2) display_pos = 0;
66 }
67
```

图 5-5 数字显示模块设计（代码）

(1) 延时函数采用双层循环实现；

(2) 数码管显示采用动态扫描方式：

- ① 先消隐避免残影；
- ② 选择要显示的位（个位（1）/十位（0））；
- ③ 输出对应段码；
- ④ 更新显示位置指针。

5.5 按键模块设计

```

68 // 按键处理函数 — 1表示短按, 2表示长按, 0表示无按键
69 unsigned char key_process() {
70     //static 使变量成为静态变量, 函数退出后仍保留值
71     static unsigned char key_state = 0; //记录按键的当前状态
72     static unsigned int press_time = 0; //记录按键持续按下的时间
73     unsigned char result = 0; //存储本次按键处理的结果
74
75     switch(key_state) {
76         case 0: // 等待按键按下
77             if(KEY_SPEED == 0) {
78                 key_state = 1; // 进入消抖状态
79                 press_time = 0;
80             }
81             break;
82
83         case 1: // 消抖状态
84             delay_ms(20);
85             if(KEY_SPEED == 0) {
86                 key_state = 2; // 确认按下, 进入计时状态
87                 isChangingSpeed = 1; // 标记开始速度调节
88             } else {
89                 key_state = 0; // 抖动, 返回等待状态
90             }
91             break;
92
93         case 2: // 计时状态
94             delay_ms(10);
95             press_time++;
96
97             if(KEY_SPEED != 0) {
98                 // 按键释放
99                 if(press_time < 50) { // 短按 (<500ms)
100                     result = 1;
101                     isChangingSpeed = 0; // 短按不切换速度
102                 } else { // 长按释放
103                     result = 2;
104                     isRunning = 1; // 长按释放后开始流水灯
105                 }
106                 key_state = 0;
107             } else if(press_time >= 50) { // 长按 (>=500ms)
108                 key_state = 3; // 进入等待释放状态
109             }
110             break;
111
112         case 3: // 等待释放状态
113             if(KEY_SPEED != 0) {
114                 result = 2; // 长按释放
115                 key_state = 0;
116                 isChangingSpeed = 0; // 结束速度调节
117                 isRunning = 1; // 长按释放后开始流水灯
118             }
119             break;
120     }
121
122     return result;
123 }
124

```

图 5-6 按键模块设计（代码）

5.6 流水灯模块设计

```
125 // 流水灯控制函数(无延时)
126 void led_flow_no_delay() {
127     // 首次启动时特殊处理
128     if(firstStart) {
129         led_pos = 0;           // 确保从第一个LED开始
130         count = 1;            // 计数设为1(显示01)
131         firstStart = 0;       // 清除首次启动标志
132     } else {
133         // 更新LED位置
134         led_pos++;
135         if(led_pos >= 8) {
136             led_pos = 0;
137         }
138
139         // 每个LED点亮时计数加1
140         count++;
141         if(count > 99) count = 0; // 计数范围0-99
142     }
143
144     // 熄灭所有LED
145     LED_PORT = 0xFF;
146     // 点亮当前LED
147     LED_PORT &= ~(0x01 << led_pos);
148 }
```

图 5-7 流水灯模块设计（代码）

- (1) 使用移位操作控制 LED 位置：0x01 << led_pos 生成对应位的掩码；
- (2) 每次流水灯移动时更新计数，范围限制在 0-99；
- (3) 首次启动时需按下按钮“开始”才会执行流水灯程序并强制从第一个 LED 开始，避免随机位置造成的不美观现象。

5.7 定时器初始化与中断服务函数设计

```

150 // 定时器0初始化函数
151 void Timer0_Init() {
152     TMOD |= 0x01; // 设置定时器0为模式1
153     TH0 = 0xFC;   // 定时1ms(12MHz晶振)
154     TL0 = 0x66;
155     ET0 = 1;      // 使能定时器中断
156     TR0 = 1;      // 启动定时器
157     EA = 1;       // 开启总中断
158 }
159
160 // 定时器0中断服务函数
161 void Timer0_ISR() interrupt 1 {
162     // 重新加载初值
163     TH0 = 0xFC;
164     TL0 = 0x66;
165
166     // 数码管扫描显示
167     display_no_delay();
168
169     // 状态显示计时
170     if(show_status) {
171         status_timer++;
172         if(status_timer >= 500) { // 500ms后关闭状态显示
173             show_status = 0;
174         }
175     }
176
177     // LED流水灯定时控制
178     if(isRunning && !show_status && !isChangingSpeed) {
179         led_timer++;
180         if(led_timer >= delay_time[speed_level-1]) {
181             led_timer = 0;
182             led_flow_no_delay();
183         }
184     }
185 }
186
187 // 显示状态函数
188 void show_status_code(unsigned char code1, unsigned char code2) {
189     // 确保传入的段码索引在有效范围内
190     if(code1 > 15) code1 = 0; // 限制在0-15范围内
191     if(code2 > 15) code2 = 0; // 限制在0-15范围内
192
193     display_buffer[0] = code1;
194     display_buffer[1] = code2;
195     show_status = 1;
196     status_timer = 0;
197 }

```

图 5-8 定时器初始化与中断服务函数设计（代码）

(1) 定时器配置为 1ms 中断：

- ① 模式 1：16 位定时器
- ② 初值计算： $(65536 - 1000) = 0xFC66$

(2) 中断服务函数主要执行：

- ① 数码管动态扫描（每 1ms 刷新一位）
- ② 状态显示计时控制
- ③ 流水灯定时控制（根据当前速度等级）

第6章 功能实现

6.1 系统功能实现

单片机芯片集成在最小系统板上，引脚与各执行模块相连，实现设备控制。依据按键接收到的控制指令，按照预定程序进行数据处理，控制流水灯设备，同时将流水灯状态状态信息通过数码管展示。

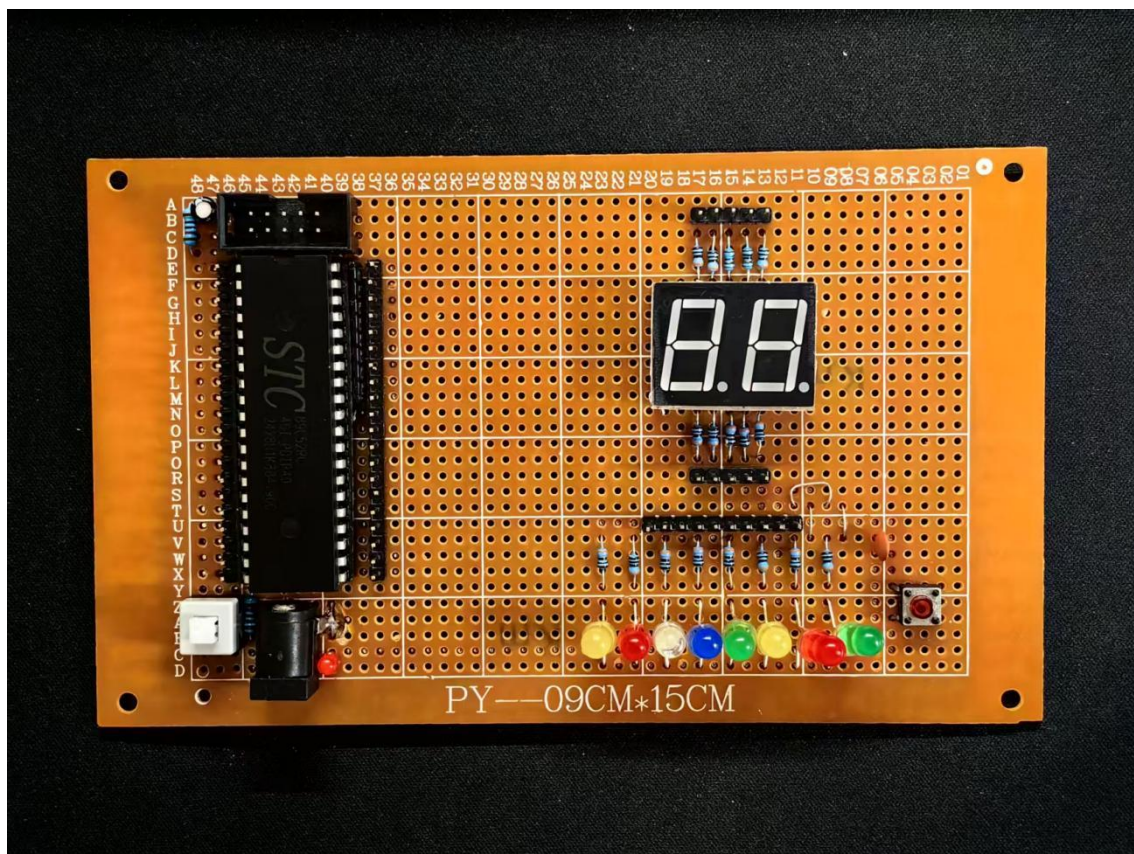


图 6-1 实物图展示

6.2 数字显示功能实现

本次实验中的数字显示模块与 STC89c52RC 单片机通过串口相连，中间串联电阻作为保护电路。

根据流水灯状态展示相应标志，流动时显示流动次数（0-99），按键控制流水灯状态时，数码管显示对应标志【起始状态：开始（AC），停止（S0）；速度状态：慢速（A1），中速（A2），快速（A3）】。

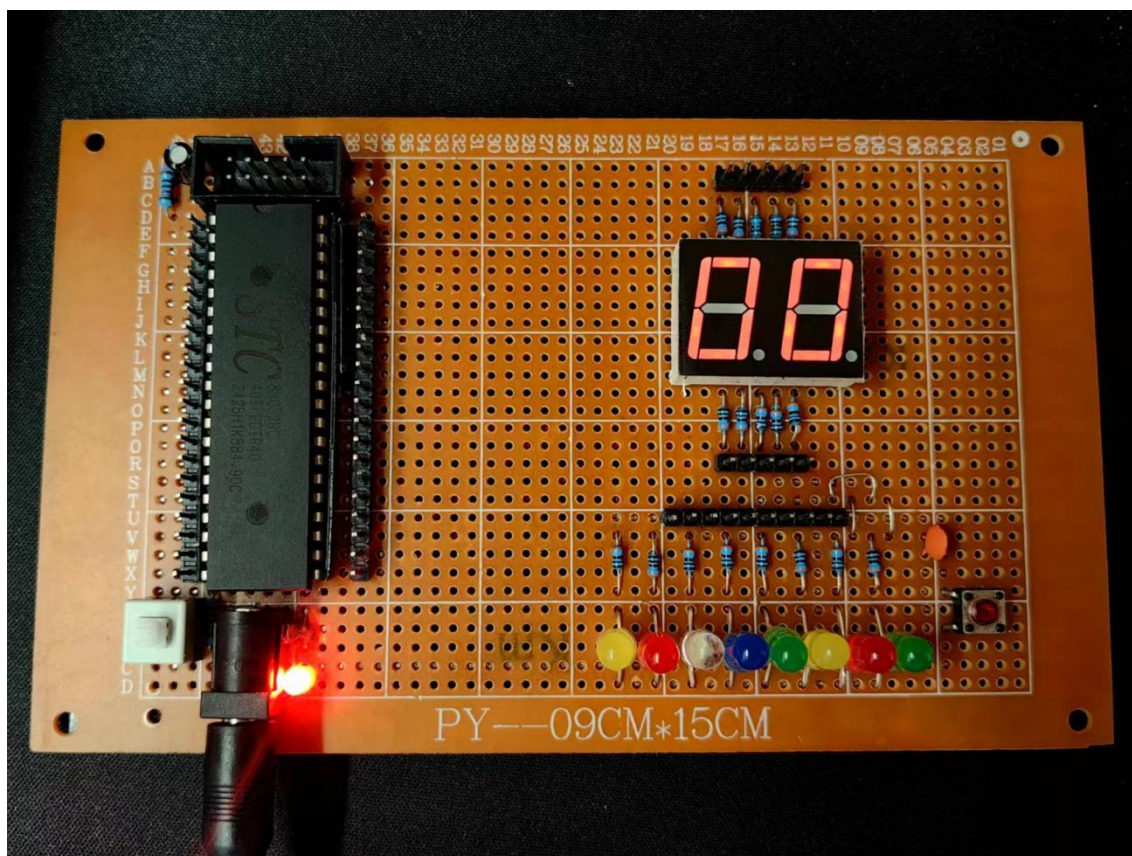


图 6-2 开机显示【最小数（00）】（实物图）

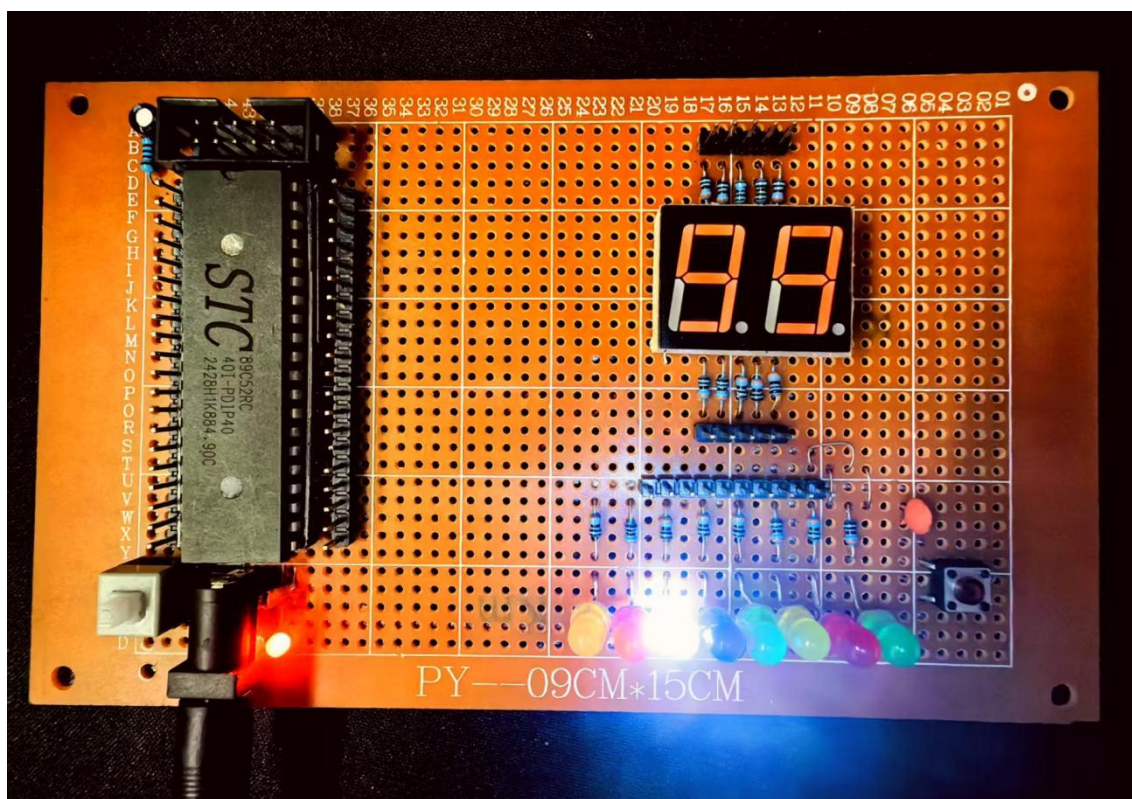


图 6-3 最大值（99）（实物图）

6.3 按键功能实现

本次实验中的按键模块与 STC89c52RC 单片机通过串口相连，中间并连电容作为防抖电路。

按键短按控制流水灯开始（AC）/停止（S0）；长按控制流水灯速度状态：慢速（A1）/中速（A2）/快速（A3），长按时数码管显示当前速度状态标志，松手后显示改变后速度状态标志。

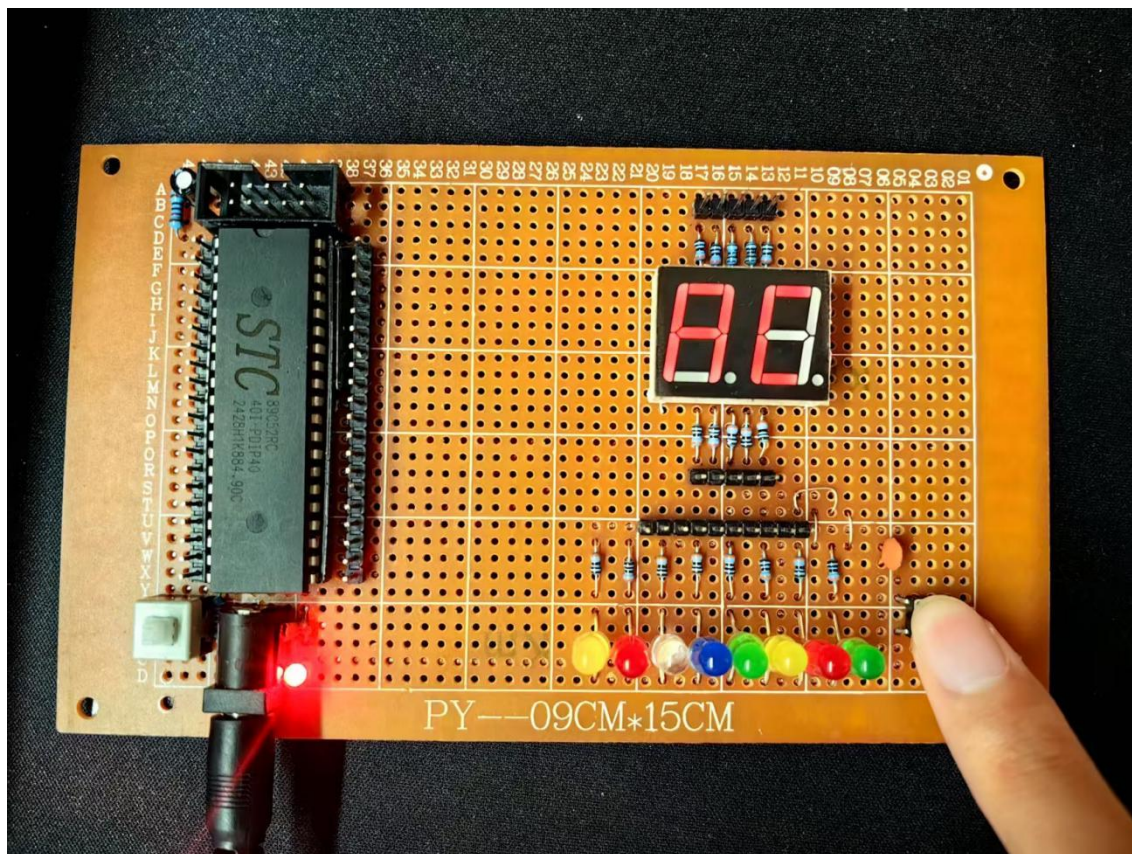


图 6-4 单击控制开始（AC）（实物图）

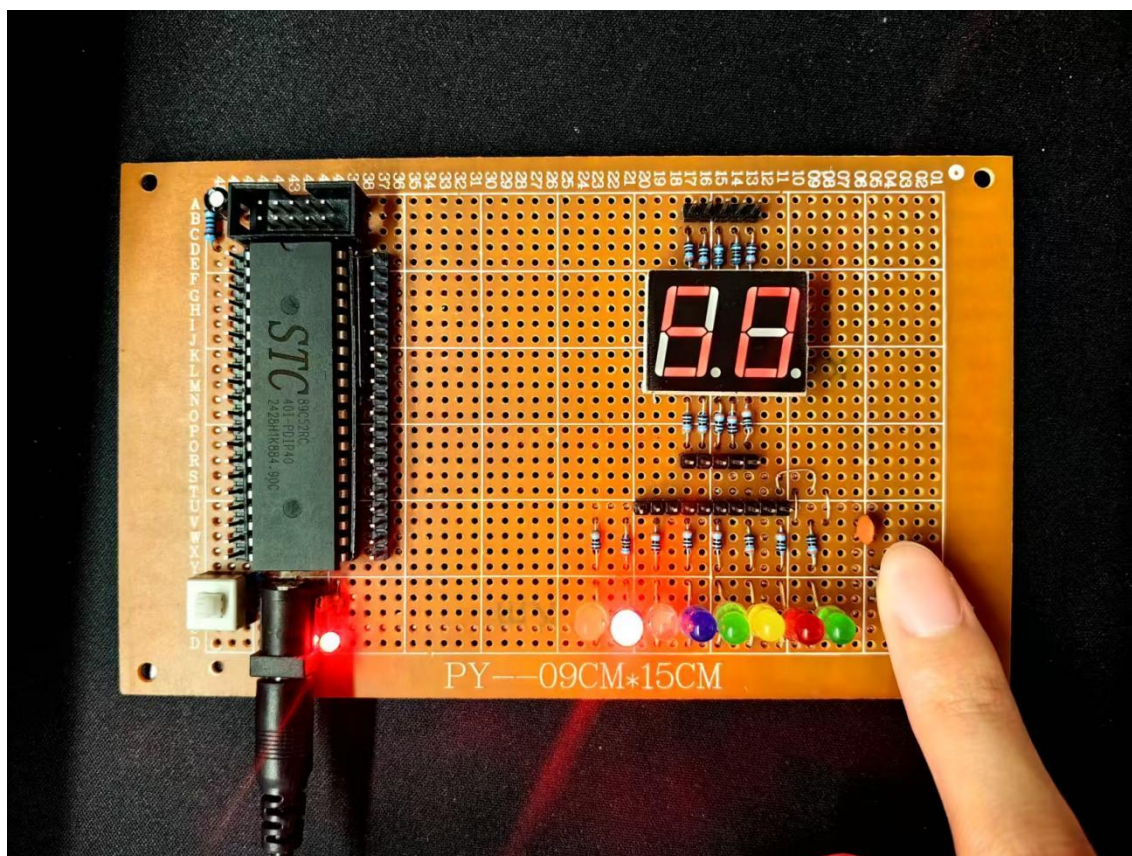


图 6-5 单击控制停止 (SO) (实物图)

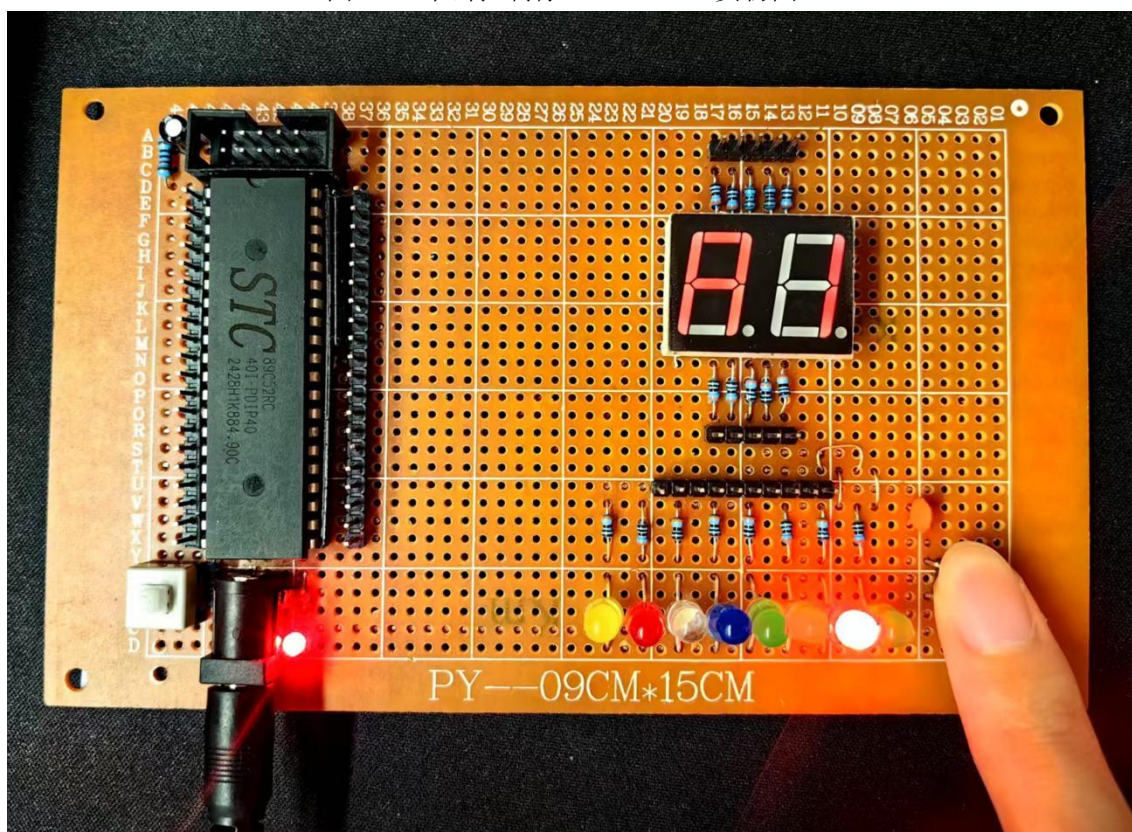


图 6-6 长按控制速度切换【慢 (A1)】 (实物图)

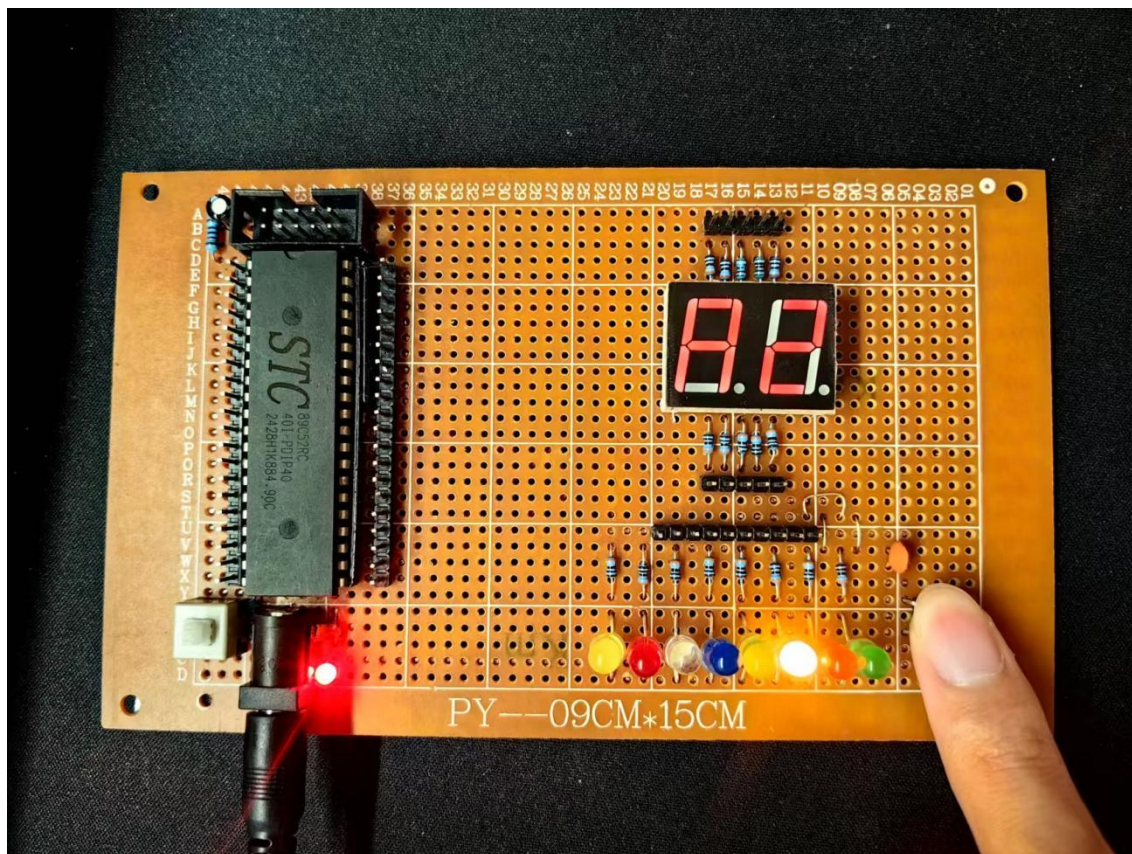


图 6-7 长按控制速度切换【中 (A2)】(实物图)

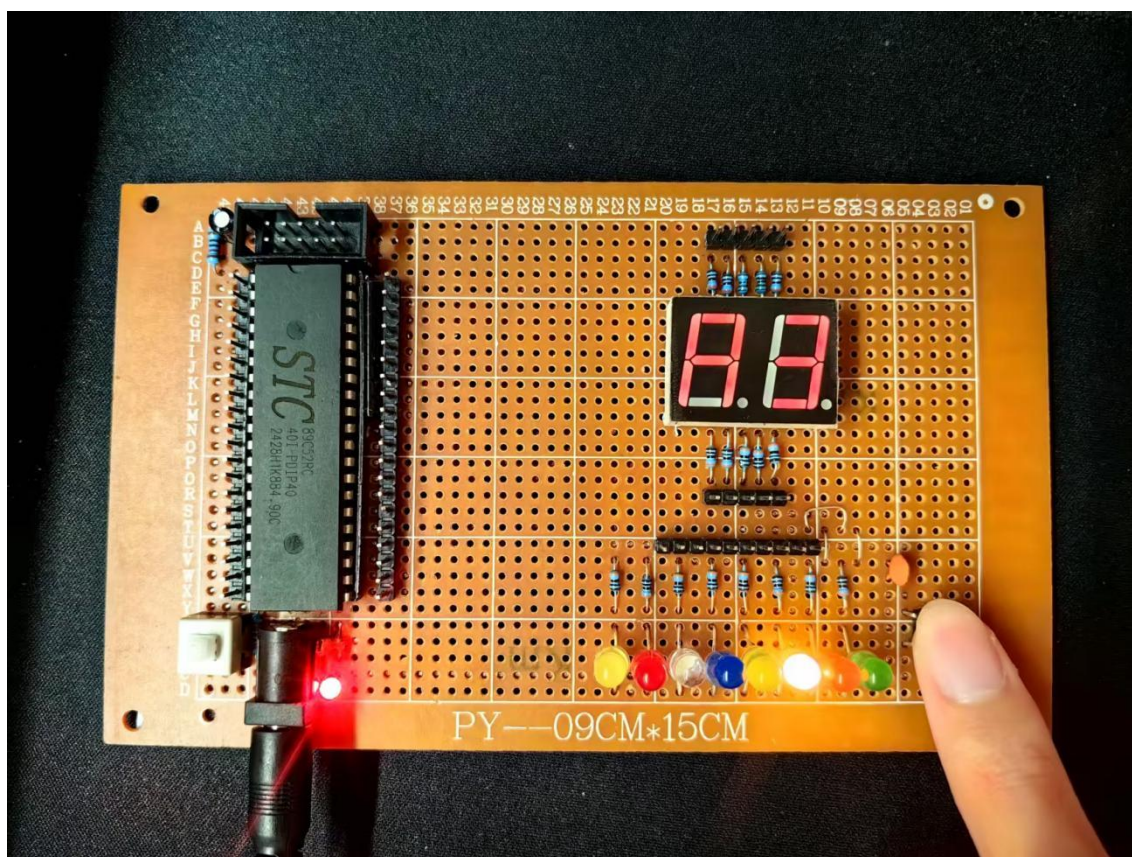


图 6-8 长按控制速度切换【快 (A3)】(实物图)

6.4 流水灯功能实现

本次实验中的流水灯模块与 STC89c52RC 单片机通过串口相连，中间串联电阻作为保护电路。

流水灯开始与停止，速度状态变换均通过按键向单片机发出对应信号，再由单片机处理后输出相应信号实现，流水灯方向为从左至右，随流水灯流动，数码管记数依次加 1，记满后从 00 开始新一轮记数。

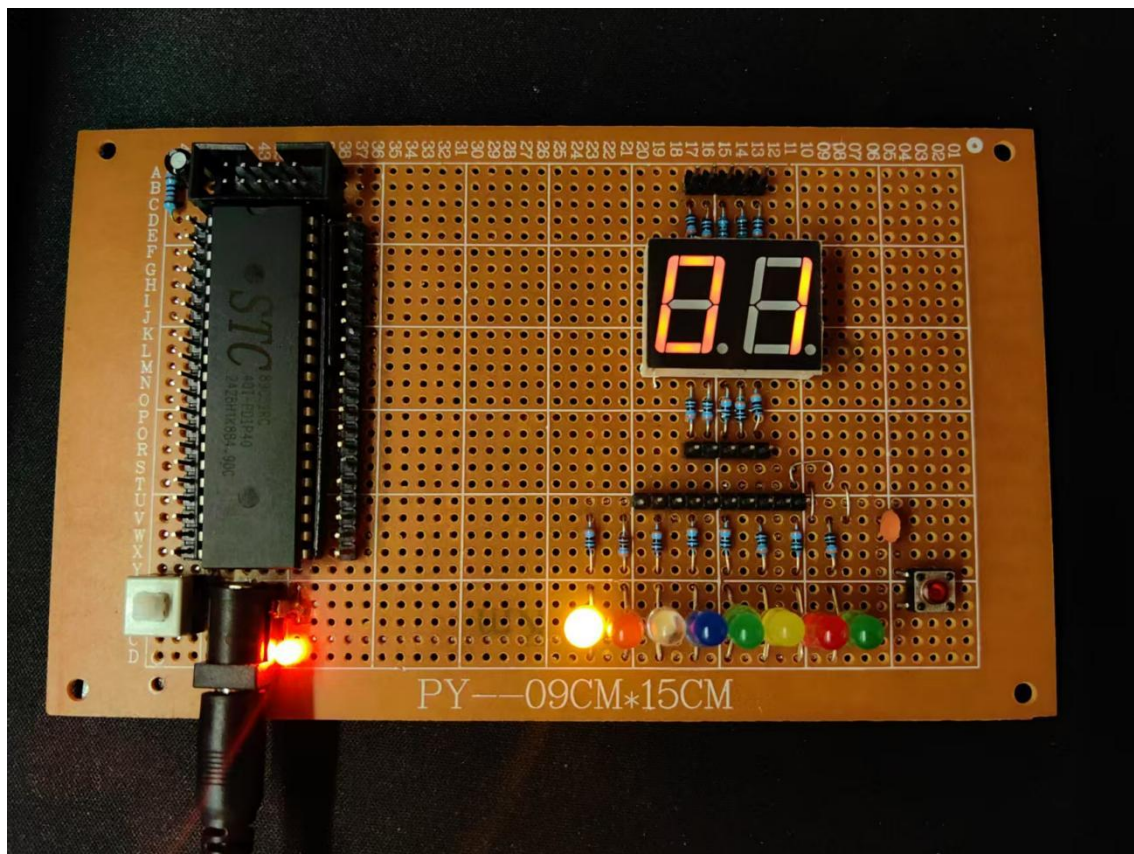


图 6-9 流水灯展示（实物图）

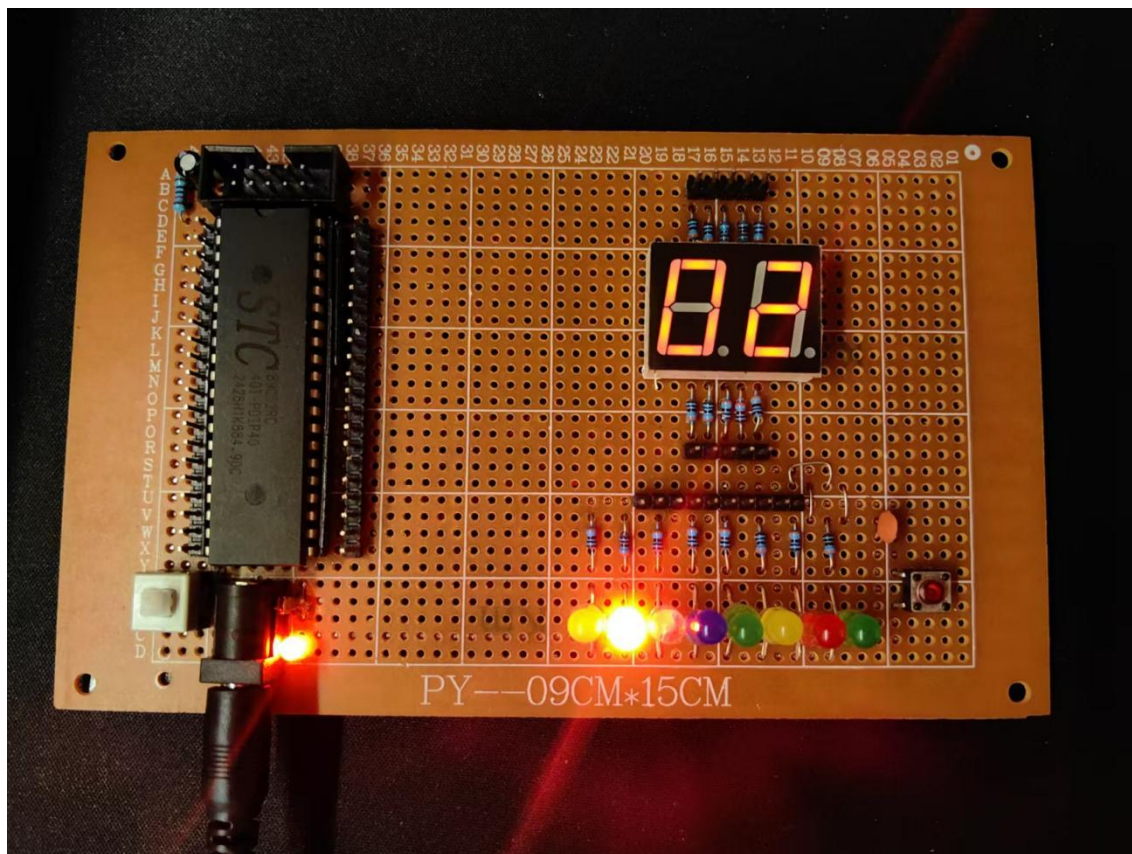


图 6-10 流水灯展示（实物图）

第 7 章 系统测试

7.1 测试目的

系统测试是为了全面评估设计成果，保障系统在实际应用场景中稳定、可靠地运行，满足课程设计计划书的功能与性能要求。

验证系统功能完整性。对系统的核心功能，如流水灯、按键控制、数码管显示等进行逐一测试，确认系统能否按设计意图，准确感应流水灯数据，及时响应并执行用户指令。

性能测试。通过模拟多设备同时运行、长时间连续工作等复杂场景，测试系统响应速度、数据传输的准确性和稳定性，评估系统的运行效率与抗干扰能力，确保系统在高负载下也能正常工作。

兼容性测试。验证系统各模块间以及与外部设备、软件的兼容性，避免因兼容性问题导致的系统故障。

7.2 功能测试

7.2.1 硬件测试

- (1) 电源测试：检查电源电路输出是否稳定在+5V。
- (2) LED 测试：逐个测试 LED 是否正常点亮，检查限流电阻选择是否合适。
- (3) 数码管测试：测试数码管的段选和位选信号是否正常，显示是否清晰。
- (4) 按键测试：测试按键是否能够正常触发，消抖处理是否有效。

7.2.2 软件测试

- (1) 流水灯功能测试：验证流水灯是否能够按照预期顺序点亮和熄灭，循环是否正常。
- (2) 计数功能测试：验证数码管是否能够正确显示流水灯的计数，计数范围是否在 0-99 之间，达到 99 后是否自动归零。
- (3) 速度控制功能测试：验证按键是否能够正常切换流水灯的速度档位，数码管是否能够正确显示当前速度档位。
- (4) 状态显示测试：验证短按和长按按键时，数码管是否能够正确显示相应的状态信息，状态信息是否在 500ms 后自动消失。

7.2.3 调试过程

- (1) 数码管显示闪烁问题：通过调整定时器中断频率和扫描方式，解决了数码管显示

闪烁的问题。

(2) 按键抖动问题：通过软件消抖和状态机设计，有效解决了按键抖动导致的误触发问题。

(3) 首次启动显示异常问题：通过添加首次启动标志和特殊处理，确保了首次启动时流水灯从第一个 LED 开始，计数显示正确。

结论

本文通过循环流水灯计数器的设计与实现，具体论述了 51 单片机最小系统搭建、硬件电路设计及软件功能编程的方法。完成的具体工作如下：

(1) 详细设计并搭建了包含电源、时钟和复位电路的 51 单片机最小系统，完成 8 个 LED 驱动电路、数码管动态扫描电路及独立按键消抖电路的设计，分析了各电路的工作原理与参数计算方法；

(2) 通过模块化编程实现了流水灯循环显示、实时计数、速度档位调节等功能，利用定时器中断、状态机等技术优化系统性能，为单片机基础应用开发提供实践参考；

(3) 完成软硬件联调与功能测试，验证了系统在流水灯控制、数码管显示及按键响应方面的稳定性，实现了良好的人机交互体验。

通过本次设计，深刻体会到单片机系统开发中软硬件协同设计的重要性，掌握了从需求分析到系统实现的完整流程，为嵌入式系统开发积累了实践经验。

由于设计时间和技术水平的限制，系统仍存在以下不足：硬件方面，未设计电源稳压及抗干扰电路，在复杂电磁环境下可能影响系统稳定性；软件层面，功能扩展性不足，仅实现基础功能，未添加流水灯花样切换、方向控制等高级功能。此外，系统缺乏错误检测与故障提示机制，用户使用时难以快速定位问题。

因此，后续工作将围绕优化硬件电路抗干扰能力、完善软件功能模块展开，计划增加低功耗模式、数据存储及异常状态报警等功能，进一步提升系统的实用性和可靠性。

参考文献

- [1] 林立, 张俊亮. 单片机原理及应用: 基于 Proteus 仿真. 北京: 电子工业出版社, 2022. 4
- [2] 李航. 基于 Proteus 软件设计流水灯系统单片机硬件电路[J]. 上海电气技术, 2020, 13(01): 51-54.
- [3] 谭艳春, 朱又敏, 刘目磊. 基于 KeilC51 和 Proteus 花样流水灯系统的设计[J]. 软件工程, 2018, 21(11): 14-16. DOI:10.19644/j.cnki.issn2096-1472. 2018. 11. 004.
- [4] 杜洪林, 周绍平. C 语言与汇编语言在单片机编程中的应用[J]. 江西科学, 2019, 37(03): 446-451. DOI:10.13990/j.issn1001-3679. 2019. 03. 025.
- [5] 马花萍. 基于单片机控制的 LED 流水灯设计[J]. 机电信息, 2016, (36): 114-115. DOI:10.19514/j.cnki.cn32-1628/tm. 2016. 36. 066.
- [6] 张成法, 赵培元. 基于 Proteus 的单片机流水灯仿真设计[J]. 智能城市, 2016, 2(10): 61. DOI:10.19301/j.cnki.zncs. 2016. 10. 049.

附录 A：循环流水灯计数器设计关键代码

```
#include<reg52.h>
#include<intrins.h>

#define LED_PORT P1
#define SEG_PORT P0
#define BIT_PORT P2

sbit KEY_SPEED = P3^2;

void delay_ms(unsigned int ms);
void display_no_delay(void);
unsigned char key_process(void);
void led_flow_no_delay(void);
void Timer0_Init(void);
void Timer0_ISR(void);
void show_status_code(unsigned char code1, unsigned char code2);

unsigned char led_pos = 0;
unsigned char count = 0;
unsigned char speed_level = 2;
unsigned int delay_time[] = {500, 300, 150};
unsigned char code seg_table[] = {
    0x3F, 0x06, 0x5B, 0x4F,
    0x66, 0x6D, 0x7D, 0x07,
    0x7F, 0x6F, 0x77, 0x7C,
    0x39, 0x5E, 0x79, 0x71
};

bit isRunning = 0;
bit show_status = 0;
unsigned int status_timer = 0;
bit isChangingSpeed = 0;
bit firstStart = 1;
```

```
unsigned char display_buffer[2] = {0, 0};
unsigned char display_pos = 0;
unsigned int led_timer = 0;

void delay_ms(unsigned int ms) {
    unsigned int i, j;
    for(i = 0; i < ms; i++)
        for(j = 0; j < 100; j++);
}

void display_no_delay() {
    SEG_PORT = 0x00;
    switch(display_pos) {
        case 0: BIT_PORT = 0x01; break;
        case 1: BIT_PORT = 0x02; break;
    }
    SEG_PORT = seg_table[display_buffer[display_pos]];

    display_pos++;
    if(display_pos >= 2) display_pos = 0;
}

unsigned char key_process() {
    static unsigned char key_state = 0;
    static unsigned int press_time = 0;
    unsigned char result = 0;

    switch(key_state) {
        case 0:
            if(KEY_SPEED == 0) {
                key_state = 1;
                press_time = 0;
            }
            break;
    }
}
```

```
case 1:
    delay_ms(20);
    if(KEY_SPEED == 0) {
        key_state = 2;
        isChangingSpeed = 1;
    } else {
        key_state = 0;
    }
    break;

case 2:
    delay_ms(10);
    press_time++;

    if(KEY_SPEED != 0) {
        if(press_time < 50) {
            result = 1;
            isChangingSpeed = 0;
        } else {
            result = 2;
            isRunning = 1;
            speed_level++;
            if(speed_level > 3) speed_level = 1;
            show_status_code(speed_level, 0xA);
        }
        key_state = 0;
    } else if(press_time >= 50) {
        show_status_code(speed_level, 0xA);
        key_state = 3;
    }
    break;

case 3:
    if(KEY_SPEED != 0) {
        result = 2;
```



```
        key_state = 0;
        isChangingSpeed = 0;
        isRunning = 1;
        speed_level++;
        if(speed_level > 3) speed_level = 1;
        show_status_code(speed_level, 0xA);
    } else {
        show_status_code(speed_level, 0xA);
    }
    break;
}

return result;
}

void led_flow_no_delay() {
    if(firstStart) {
        led_pos = 0;
        count = 1;
        firstStart = 0;
    } else {
        led_pos++;
        if(led_pos >= 8) {
            led_pos = 0;
        }

        count++;
        if(count > 99) count = 0;
    }

    LED_PORT = 0xFF;
    LED_PORT &= ~(0x01 << led_pos);
}

void Timer0_Init() {
```

```
    TMOD |= 0x01;
    TH0 = 0xFC;
    TL0 = 0x66;
    ET0 = 1;
    TR0 = 1;
    EA = 1;
}

void Timer0_ISR() interrupt 1 {
    TH0 = 0xFC;
    TL0 = 0x66;

    display_no_delay();

    if(show_status) {
        status_timer++;
        if(status_timer >= 500) {
            show_status = 0;
        }
    }

    if(isRunning && !show_status && !isChangingSpeed) {
        led_timer++;
        if(led_timer >= delay_time[speed_level-1]) {
            led_timer = 0;
            led_flow_no_delay();
        }
    }
}

void show_status_code(unsigned char code1, unsigned char code2) {
    if(code1 > 15) code1 = 0;
    if(code2 > 15) code2 = 0;

    display_buffer[0] = code1;
```

```
    display_buffer[1] = code2;
    show_status = 1;
    status_timer = 0;
}

void main() {
    unsigned char key_result;

    LED_PORT = 0xFF;
    Timer0_Init();

    while(1) {
        key_result = key_process();

        if(key_result == 1) {
            isRunning = !isRunning;
            if(isRunning) {
                show_status_code(0xC, 0xA);
            } else {
                show_status_code(0x0, 0x5);
            }
        }

        if(!show_status && !isChangingSpeed) {
            display_buffer[0] = count % 10;
            display_buffer[1] = count / 10;
        }
    }
}
```

单片机应用课程设计 设计成绩评定表

序号	评价指标	满分值	得 分
1	课程设计过程中的学习态度认真、按时完成相应的任务	10	
2	作品满足基本需求，答辩过程中对所实现功能讲解及展示清晰	30	
3	答辩过程中对老师提出的问题回答全面、准确	15	
4	报告中需求描述全面、准确，与实际相符，系统整体方案及功能设计合理	10	
5	各功能模块的设计流程图正确	10	
6	功能展示效果与实际作品一致，且满足设计需求	10	
7	对作品进行功能扩展与创新	5	
8	报告行文通顺，格式规范，条理清晰	10	
总 评			

注：优：≥90 分 良：80~89 分 中：70~79 分 及格：60~69 分 不及格：<60 分

教师评语等级（ ）：

优：能很好地完成课设任务，报告能对课设内容进行全面、系统的总结，并能运用学过的理论知识对某些问题加以分析，态度端正，期间无违纪行为。

良：能较好地完成课设任务，报告能对课设内容进行比较全面、系统的总结，态度端正，期间无违纪行为。

中：达到大纲中规定的主要要求，报告能对课设内容进行比较全面的总结，态度端正，期间无违纪行为。

及格：态度基本端正，完成了课设的主要任务，能够完成课设报告，内容基本正确；期间虽然有较轻的违纪行为，但能够深刻认识，及时纠正。

不及格：凡具备下列条件之一者，均以不及格论处：

- 1、未达到大纲中规定的基本要求，课设报告马虎潦草或内容有明显错误。
- 2、未能参加的时间超过全部时间的三分之一以上者。
- 3、实践中有违纪行为，经教育不改；或有严重违纪行为；或发生重大事故者，取消其课设资格，成绩作不及格处理。

指导教师(签章)：

年 月 日